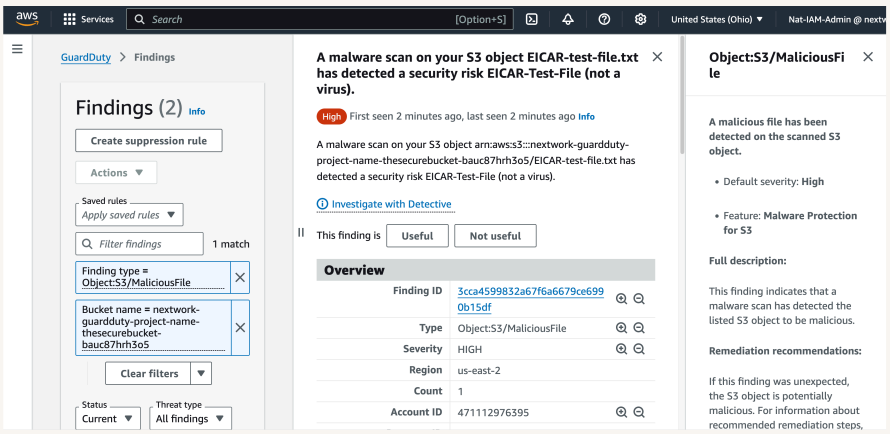
NextWork.org

Threat Detection with GuardDuty

NA Natasha Ong



NA
Natasha Ong
NextWork Student
NextWork.org

Introducing Today's Project!

Tools and concepts

The services we used were GuardDuty, CloudFormation, S3, and CloudShell. Key concepts we learnt include SQL + command injection, using Linux commands like wget, cat and jq, and malware protection!

Project reflection

This project took us approximately 2.5 hours including demo time. The most challenging part was getting access to the victim's AWS environment using a profile. It was most rewarding to reveal the victim's protected file as the attacker.

We did this project today to learn about threat detection and GuardDuty + techniques like SQL/command injection. This project certainly met our expectations and was a good challenge!

NA

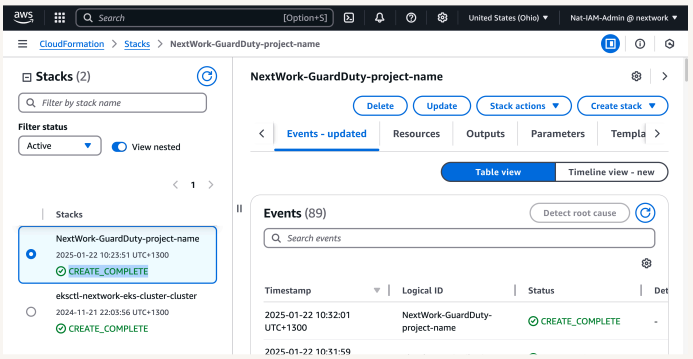
Natasha Ong
NextWork Student

Project Setup

To set up for this project, we deployed a CloudFormation template that launches an insecure web app (OWASP Juice Shop). The three main components are the web app infrastructure, an S3 bucket and GuardDuty protecting our environment.

The web app deployed is called OWASP Juice Shop. To practice our GuardDuty skills, we will attack the Juice Shop, and then visit the GuardDuty console to detect and analyze its findings - does it pick up on our attacks to our web app?

GuardDuty is an AI-powered threat detection service, which means it is designed to help us find and security attacks or vulnerabilities that affects our AWS resources/environment. Once it detects something unusual, it's up to us to investigate.



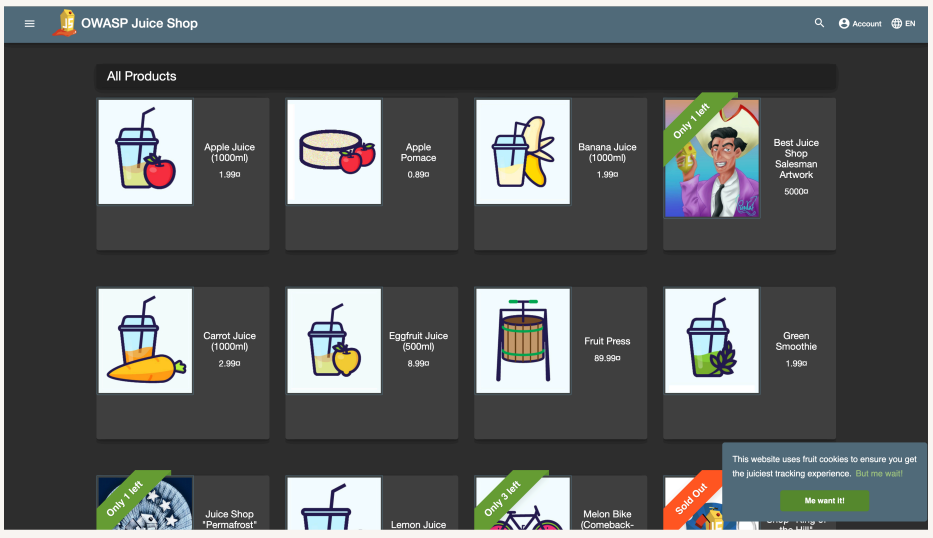
NA

Natasha Ong
NextWork Student

SQL Injection

The first attack we performed on the web app is SQL injection, which means injecting malicious SQL code that manipulates a result from our web app. SQL injection is a security risk because it can let attackers bypass logins, or delete/edit data.

My SQL injection attack involved entering the code `' or 1=1;--` into the email field of the web app's login page. This means the login query will always evaluate to true (i.e. our database is manipulated into telling our web app this login exists).

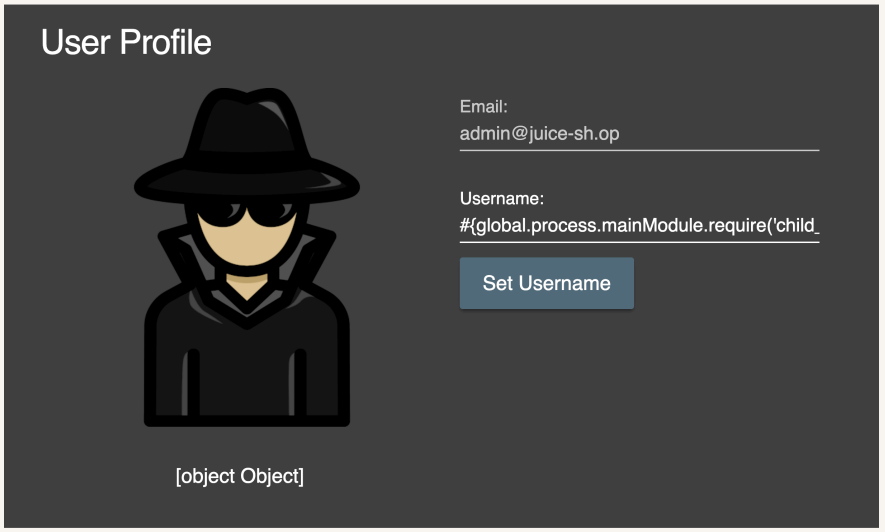


NA
Natasha Ong
NextWork Student
NextWork.org

Cross Site Scripting

Next, we used command injection, which manipulates the web app's web server to run code that has been entered e.g. in a form. The Juice Shop web app is vulnerable to this because it does not sanitize user inputs i.e. does not block scripts.

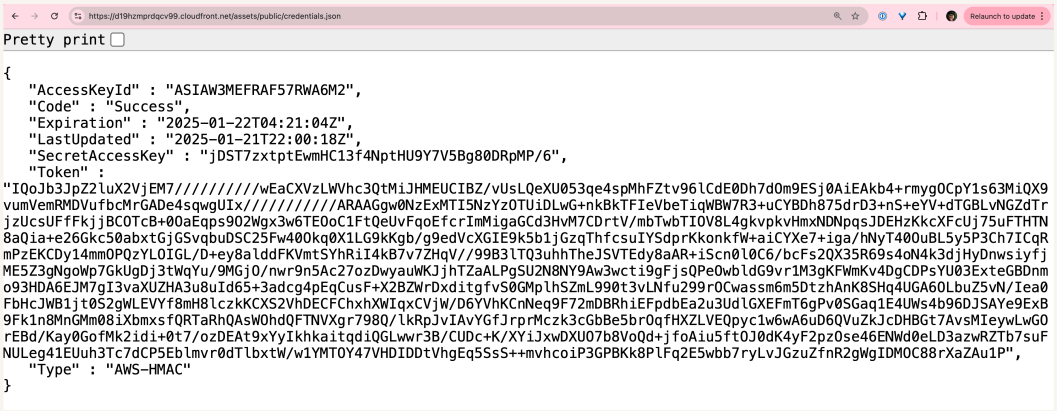
To run command injection, we entered JavaScript code in the username field of the web app's admin console. This script wil tell our web server to expose the server's IAM credentials and save them in a publicly accessible JSON file.



NA
Natasha Ong
NextWork Student

Attack Verification

To verify the attack's success, we visited the publicly exposed credentials file. This page showed us access keys that represents our EC2 instance's access to the developer's AWS environment. We can use those keys to get the same level of access.



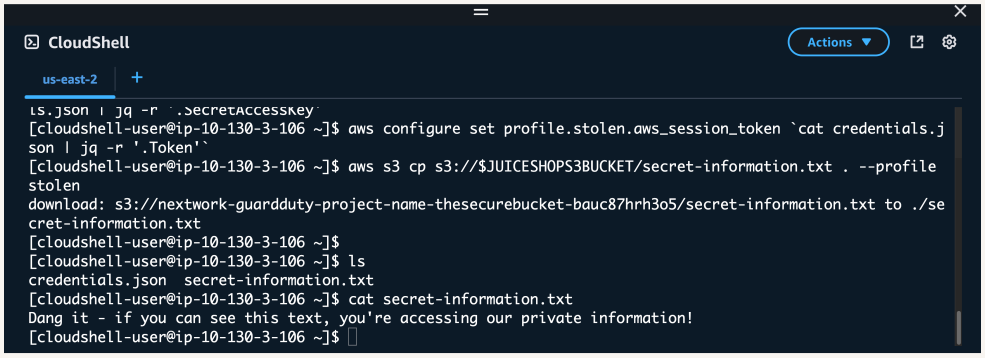
NA
Natasha Ong
NextWork Student

Using CloudShell for Advanced Attacks

The attack continues in CloudShell, because this is a command-line tool we can use to run AWS commands that uses the credentials we've stolen. CloudShell is now our medium for doing suspicious things like stealing data from an S3 bucket.

In CloudShell, we used wget to download the exposed credentials file into our CloudShell environment. Next, we ran a command using cat and jq to read the downloaded file and format it nicely - so the credentials (in JSON) are easy to understand.

We then set up a new profile using all of the stolen credentials. We had to create a new profile because the hacker doesn't inherently have access to the victim's AWS environment. We'll need to use the profile to switch permission settings.



NA

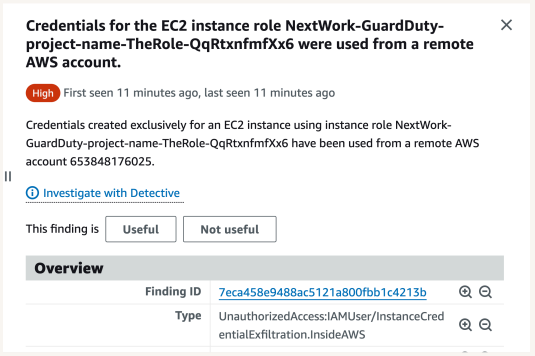
Natasha Ong
NextWork Student

GuardDuty's Findings

After performing the attack, GuardDuty reported a finding within 15 minutes. Findings are notifications from GuardDuty that something suspicious has happened, and they give you additional details about the who/what/when of the attack.

GuardDuty's finding was called UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.InsideAWS, which means credentials belonging to my EC2 instance were being used in another account. Anomaly detection was used because this was unusual behaviour

GuardDuty's detailed finding reported the S3 bucket was affected; the action that was done using the stolen credentials (GetObject); and the EC2 instance whose credentials were leaked. The IP address + location of the actor was also available.



NA

Natasha Ong
NextWork Student

Extra: Malware Protection

For our project extension, we enabled Malware Protection for S3. Malware is file that containts threats e.g. opening the file will cause a data breach or a deletion of resources.

To test Malware Protection, we uploaded an EICAR test file into a protected bucket. The uploaded file won't actually cause damage because the test file is only designed to alert antivirus software.

Once we uploaded the malware, GuardDuty instantly triggered a finding called Object:S3/MaliciousFile. This verified that GuardDuty could successfully detect malware. It also mentioned that the threat type is EICAR-Test-File (which means not a virus).